一. A computer system has one CPU, one input processor and one printer. Two processes A and B enter the system sequentially, and A is scheduled by the CPU scheduler at first. The execution tracks of A and B are as follows:

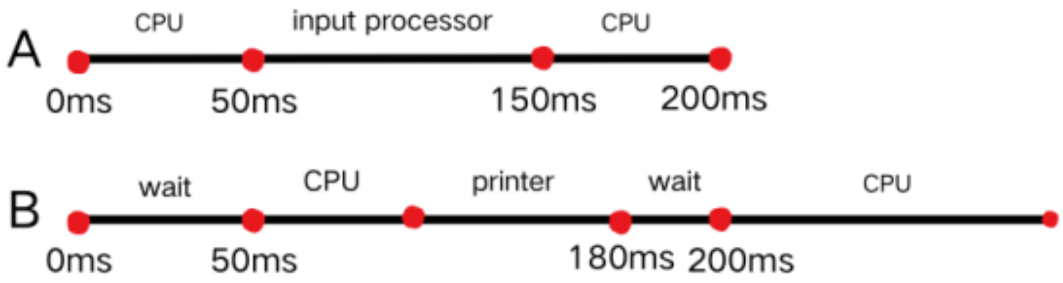
A: CPU burst lasting 50ms, then I/O burst of 100ms on the input processor, and then CPU burst lasting 50ms, exiting

B: CPU burst lasting 50ms, then I/O burst of 80ms on the printer, and then CPU burst lasting 100ms, exiting

(1)Draw the Gantt chart to describe the resource usage of A and B on the CPU, the input processor and the printer.

(2)Calculate the waiting time and turnaround time for process A and B respectively.

(1) Gantt chart A and B



(2)

Process A:

Waiting time = 0 ms

Turnaround time = 200 ms

Process B:

Waiting time = 70 ms (50ms initial wait + 20ms wait at 180ms)

Turnaround time = 300 ms

二． Three processes P1、P2 and P3 using a buffer having N(N>0) units mutually exclusively. For each time, P1 uses produce() to generate an non-negative integer and then uses put() to send the integer into an empty unit in the buffer; P2 uses getodd() to pick one odd number from the buffer and then uses countodd() to count the number of odd numbers; P3 uses geteven() to pick one even number frome the buffer and then uses counteven() to count the number of even numbers. Please implement the synchronization and mutual exclusion activities of these three processes using the semaphore mechanism and describe the meaning of the semaphores you defined. It requires a pseudo-code description.

Ans:

Semaphores

- mutex = 1(mutual exclusion for buffer access)
- empty = N(counts empty buffer slots)
- odd = 0(counts odd numbers in buffer)
- even = 0(counts even numbers in buffer)

Process P1:

```
while true {
    num = produce();
    wait(empty);
    wait(mutex);
    put(num);
    signal(mutex);
    if (num % 2 == 0) signal(even);
    else signal(odd);
}
```

```
Process P2:
    while true {
        wait(odd);
        wait(mutex);
        num = getodd();
        signal(mutex);
        signal(empty);
        countodd();
    }

Process P3:
    while true {
        wait(even);
        wait(mutex);
        num = geteven();
        signal(mutex);
        signal(empty);
        counteven();
    }
```

三． A file system uses 256-byte physical blocks. Each file has a directory entry giving the file name, location of the first block, length of file, and last block position. Assuming the last physical block read is 100, block 100 and the

directory entry are already in main memory.

For the following two file allocation algorithms, how many physical blocks must be read to access the specific block 600 (including the reading of block 600 itself), and why?

(1) contiguous allocation

(2) linked allocation

(1) 1

Contiguous allocation supports direct access on arbitrtary physical blocks in a file. To access physical block 600, the bolck 600 can be directory read into memory.

(2) 500

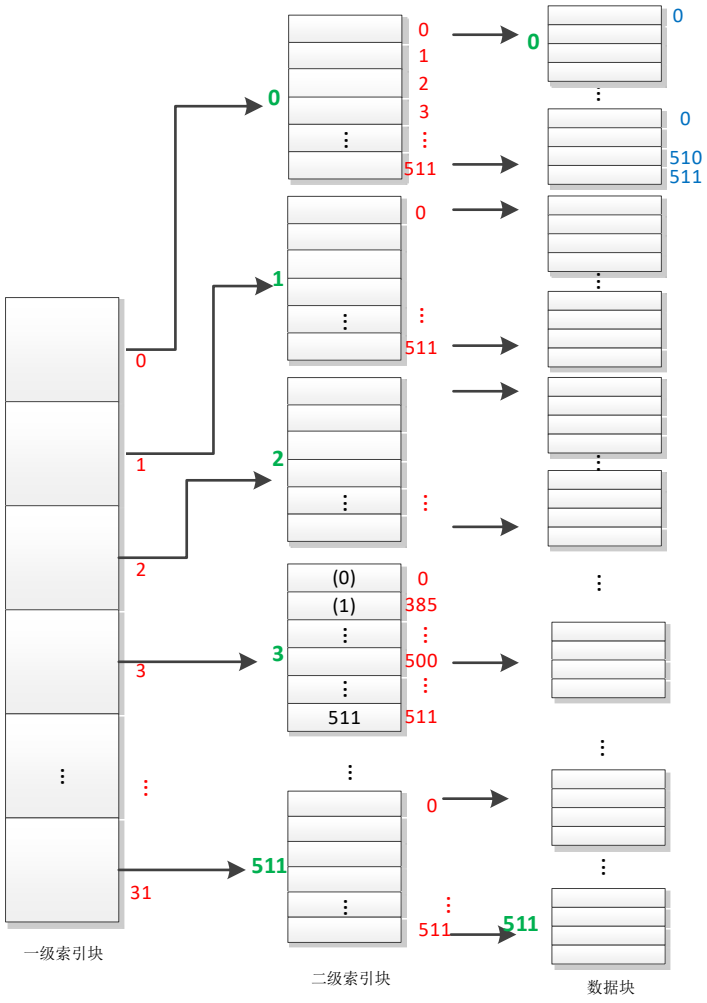
Linked allocation is used only for sequential-access files, file system can only read blocks one by one, directed by file pointers.

To find the phyical block 600 in the file, file system locates on the 101th block following the 100th block just read, and reads block 101, block 102, …, until block 600.

四. Consider a file system in which a directory entry can store up to 32 disk block addresses. For the files that is not larger than 32 blocks, the 32 addresses in the entry serves as the file's index table.

For the files that is larger than 32 blocks, each of the 32 addresses points to an indirect block that in turn points to 512 file blocks on the disk, and the size of a block is 512-bytes.

What is the largest size of a file?



32 x 512 =16384 file blocks

16384 x 512 = 8388608 bytes

or

2^5 x 2^9 x 2^9 = 2^{23} bytes